Distributed Food Ordering System - Assignment Spec

Course: Big Data 2026 (UE24CS343AB2) **Technologies:** Apache Kafka, Apache Spark, KRaft, ZeroTier. This is a transcription of the assignment page plus the team's agreed workflow, written so it can be handed to a coding assistant as the source of truth.

1. Introduction

A food-delivery platform processes a large number of orders. Order data is first published to Kafka, which acts as the distributed messaging backbone. A downstream processing service consumes these orders and uses Spark to clean and enrich them with delivery-related metrics. The processed order data is then published back to Kafka as a new topic, where another service consumes it and runs Spark analytics to understand restaurant and delivery performance. The assignment is to build this **two-stage, Kafka and Spark-based batch data pipeline** across four VMs.

2. Objective

- configure a **multi-broker Kafka cluster using KRaft**;
- understand how topics, partitions and replication work together;
- demonstrate how Kafka maintains availability of replicated data when a broker fails;
- build Kafka producers and consumers that move data between independent processing stages;
- use Apache Spark Batch to transform raw order data into enriched order-level metrics;
- publish processed data as a new Kafka topic for downstream consumption;
- use a second Spark Batch job to perform restaurant- and city-level analytics.
- Key idea: **Kafka provides reliable, distributed data movement; Spark provides batch computation over the data at the consumer end.**

3. System Roles

VM	Role	Responsibility
VM1	Producer 1, Spark Job 2, Consumer 2	Publishes `orders`; consumes `order_metrics`; runs Spark Job 2
VM2	KRaft Controller + Kafka Broker 1	Kafka broker and controller
VM3	Kafka Broker 2	Kafka broker
VM4	Consumer 1, Spark Job 1, Producer 2	Consumes `orders`; runs Spark Job 1; publishes `order_metrics`

VM2 and VM3 form the Kafka broker cluster, with VM2 serving as both broker and controller.

4. Data Flow Overview

Stage 1 - Order Enrichment

1. VM1 publishes the input dataset to Kafka topic `orders`.
2. VM4 consumes `orders` and stores the records on its local filesystem as `orders_landed.jsonl`.
3. VM4 runs **Spark Job 1** on `orders_landed.jsonl`.
4. Spark produces the enriched order-level records and writes them to `order_metrics.jsonl`.
5. VM4 publishes the records from `order_metrics.jsonl` to Kafka topic `order_metrics`.

Stage 2 - Operations Analytics

1. VM1 consumes `order_metrics` and stores the records on its local filesystem as `order_metrics_landed.jsonl`.
2. VM1 runs **Spark Job 2** on `order_metrics_landed.jsonl`.
3. Spark produces the final restaurant and city analytics files.

5. About the Dataset

Each team receives a unique dataset. All teams follow the same schema and processing rules, but the actual records differ.

Input schema (JSON Lines, one object per line)

```
order_id, customer_id, restaurant_id, delivery_partner_id, city,
order_timestamp, accepted_timestamp, picked_up_timestamp,
expected_delivery_timestamp, delivered_timestamp,
distance_km, order_amount, delivery_fee, status
```

Cleaning rules (Spark Job 1 must ignore records where):

- the JSON record is malformed;
- a required field is missing or invalid;
- any required timestamp is invalid;
- `distance_km <= 0`;
- `order_amount <= 0`;
- `status` is not `DELIVERED`.
- For **both** Spark Jobs 1 and 2, exact duplicate orders must be counted only once.

6. Spark Jobs to be Run

Spark Job 1 - Order Enrichment

Input: `orders_landed.jsonl`. For every valid order, calculate:

```
preparation_time_min = picked_up_timestamp - accepted_timestamp
delivery_time_min    = delivered_timestamp - picked_up_timestamp
fulfillment_time_min = delivered_timestamp - order_timestamp
delivery_speed_kmph  = distance_km / (delivery_time_min / 60)
delay_min            = max(0, delivered_timestamp - expected_delivery_timestamp)

delivery_status:  delay_min = 0 -> ON_TIME
                  delay_min > 0 -> DELAYED

delay_category:   0 min   -> ON_TIME
                  1-15    -> SLIGHT_DELAY
                  16-30   -> MODERATE_DELAY
                  >30     -> SEVERE_DELAY
```

(All time differences are in minutes.) Write the enriched records to `order_metrics.jsonl`.

Expected enriched schema

```
order_id, customer_id, restaurant_id, delivery_partner_id, city,
order_amount, delivery_fee, distance_km, preparation_time_min,
delivery_time_min, fulfillment_time_min, delivery_speed_kmph,
delay_min, delivery_status, delay_category
```

Then publish the records from this file to Kafka topic `order_metrics`.

Spark Job 2 - Operations Analytics

Input: `order_metrics_landed.jsonl`. Perform **exactly** these two analytics.

A. Restaurant Performance - group by restaurant_id

```
restaurant_id, order_count, avg_order_amount, avg_preparation_time_min,
avg_delivery_time_min, delayed_order_count, delay_rate_pct

delay_rate_pct = 100 * delayed_order_count / order_count
```

Output: `restaurant_summary.csv`

B. City Delivery Performance - group by city

```
city, order_count, avg_order_amount, avg_delivery_time_min,
delayed_order_count, delay_rate_pct
```

Output: `city_summary.csv`

All calculated numerical values must be rounded to 2 decimal places.

7. Expected Output

File / Topic	Description
orders_landed.jsonl	Records consumed by VM4
order_metrics.jsonl	Enriched records produced by Spark Job 1 on VM4; must also be published to order_metrics
order_metrics_landed.jsonl	Records consumed by VM1
restaurant_summary.csv	Final restaurant-level analytics
city_summary.csv	Final city-level analytics

8. Implementation Notes

- **Do not have Spark consume Kafka directly.** Kafka data must first be consumed into a local file, which is then used as the Spark input.
- **Do not use HDFS.** The intended landing area is the local filesystem.
- The output of Spark Job 1 must be **published to Kafka** as `order_metrics`. Do not transfer the file directly from VM4 to VM1.
- Use **2 partitions and replication factor 2** for both Kafka topics.
- `orders` and `order_metrics` must use the **same Kafka broker cluster**.
- The broker-failure test must be performed by stopping **VM3** (not VM2) and verifying that replicated partitions remain available through the surviving broker. Controller handover is out of scope for this assignment.
- Apply the specified cleaning, duplicate-handling, transformations and analytics **exactly as given**; do not add or change business rules.
- Final outputs must be generated from the team's assigned dataset and not hardcoded.

9. Team Workflow (agreed plan)

One VM per member; prepare code independently, then integrate during a scheduled online session. For now one person may build all four roles locally and the team decides ownership later.

Member / VM	Deliverables
Member 1 / VM1 - Producer 1 + Analytics	producer1.py, consumer2.py (order_metrics -> order_metrics_landed.jsonl), spark_job2.py, test outputs. Check aggregation logic and 2-decimal rounding.
Member 2 / VM2 - Broker 1 + KRaft Controller	Java + Kafka install, KRaft controller/broker config template; document broker ID, listeners, advertised addresses, controller quorum, storage format, startup and health-check commands.

Member / VM	Deliverables
Member 3 / VM3 - Broker 2	Broker-only KRaft config that joins VM2's cluster (do not init a separate cluster); commands to verify topics, partitions, replication factor, replica status; broker-failure test checklist.
Member 4 / VM4 - Consumer 1 + Spark Job 1 + Producer 2	consumer1.py (orders -> orders_landed.jsonl), spark_job1.py (validation, cleaning, dedup, metrics, order_metrics.jsonl), producer2.py (publish enriched records to order_metrics).

Shared interface contract

- Raw topic: `orders`; enriched topic: `order_metrics`; both with 2 partitions and replication factor 2.
- Files: orders_landed.jsonl, order_metrics.jsonl, order_metrics_landed.jsonl (JSON Lines); restaurant_summary.csv, city_summary.csv (CSV).
- Use exact field names everywhere (in particular `order_amount`).
- Deduplicate exact duplicate orders in both Spark jobs; round calculated numbers to 2 decimals.

Repository layout

```
distributed-food-ordering/
  producer/   (producer1.py, producer2.py)
  consumer/   (consumer1.py, consumer2.py)
  spark/      (spark_job1.py, spark_job2.py)
  kafka/      (VM2 and VM3 config notes; topic commands)
  tests/      (small synthetic JSONL fixtures)
  outputs/    (local test outputs)
  requirements.txt
  README.md
```

No passwords or machine-specific secrets in the repo; use placeholders for addresses in config templates.

Integration session checklist

- Connect all four VMs over ZeroTier (or another reachable network); verify clients reach brokers at their advertised addresses.
- Create/verify both topics with 2 partitions and replication factor 2.
- Stage 1: VM1 publishes orders -> VM4 lands, enriches, republishes order_metrics.
- Stage 2: VM1 consumes order_metrics and generates both final CSVs.
- Stop VM3; verify replicated partitions remain available through the surviving broker. Controller handover is out of scope.
- Save logs, screenshots and final outputs for submission.

10. Notes on the provided order.jsonl (observed)

The team file has 62 lines and deliberately contains dirty data that the cleaning rules must handle:

- Malformed / non-order lines: `{"order_id":"BROKEN","customer_id":"C"}` and two `invalid_order_record` lines.
- Invalid timestamps: T040005 (accepted_timestamp = "bad") and T040003 (delivered_timestamp = "not-a-timestamp"); each also has a valid version elsewhere in the file.
- Missing field: one T040002 line has no `city`.
- Bad numbers: T040001 has a copy with distance_km = -2; T040004 has a copy with order_amount = 0.
- Non-DELIVERED statuses: T04X0001 to T04X0006 (CANCELLED, PENDING, CANCELLED, FAILED, CANCELLED, REFUNDED).
- Exact duplicates: e.g. T040039, T040017, T040028 and T040006 each appear twice, identical.

- Some orders cross midnight (e.g. T040023, T040024); durations must be computed from full timestamps, not times of day.

Dataset link: the "A2 Dataset Download" URL was not included in the pasted text; the file above was uploaded directly.